目录

RMSNORM: 2
ROPE: 3
Attention: 7
ResNet: 9
激活函数： 10
FFN： 14
KV cache： 15
DataSet: 16
动态学习率： 17
梯度累积： 18
混合精度： 在计算过程中，同时使用 16 位浮点数 (FP16) 和 32 位浮点数 (FP32)。 18
① 权重备份 (Master Weights) 18
② 损失缩放 (Loss Scaling) 18
Flash attention: 18
Pre-train: 20
SFT（有监督微调）： 20

## RMSNORM:

由于在反向传播算法中：对一个 hidden layer:y=w·x

$$\frac{dl}{dw}=\frac{dl}{dy}\times\frac{dy}{dw}=\frac{dl}{dy}\times x$$

即对于参数的梯度为误差项乘于输入 x，若 x 数值偏差过大过小会导致梯度不稳定（梯度爆炸\消失）。则对于 x 进行 RMSNORM 是很必要的，即让输入的均值为 0，标准差为 1。

RMSNORM 相当于 NORM 少了减去均值的部分：

$$y_i=\frac{x_i}{\sqrt{\sum_{i=1}^{n}{x_i}^2+\epsilon}}\times\gamma$$

其中 ε 是一个接近 0 的数值用来防止出现除 0 操作，γ 为权重系数（当数据分散后对于不同数据给与的权重决定对其放大 or 缩小）。

ROPE:

**为什么需要位置编码？**

当我们想要利用 Transformer **架构**来训练一个大语言模型时

1. 首先会输入一段文本，或者通俗地讲是一句话，比如“我喜欢你”。

2. 这句话会经过已经训练好的分词器进行初步分词，转换为 token 数学向量形式；
3. 之后，token 会经过**词嵌入层（input embedding）**，其神经网络输入输出形状为（batch_size,sequence_length,d_model）转换为**稠密向量**；
4. 转换后的稠密向量 X 会输入到**自注意力机制**中，具体的运算就是

**Softmax**$(\frac{QK^T}{\sqrt{d_k}})$**V**，其中 Q(query)、K(key)、V(value) 是 X

分别与$W_q$、$W_k$、$W_v$三个矩阵进行线性投影之后的结果。

5. 那么实际上 $QK^T = W_qXX^TW_k$的运算主要依赖于原始的稠密向量 $XX^T$。而

对于一句话来说，如果不去关注每一个字的**位置信息**，只是关注字词内容本身，那么每个 token 所转换成的稠密向量 X 肯定是相同的，经过注意力机制后相同文本不同位置的语句结果也不会变化，但这是不符合实际情况的。

比如:“我喜欢你”→ [”我”，“喜欢”，“你”] → [[1,2,3],[4,5,6],[7,8,9]]

这句话如果改为”你喜欢我”→[”你”，“喜欢”，“我”]→[[7,8,9],[4,5,6],[1,2,3]]

可以看到，虽然”我喜欢你”和”你喜欢我”这两句话语义有很大的不同，但

最后转换成的稠密向量 X 在具体的值上没有变化，都是 [1,2,3,4,5,6,7,8,9]。

我们引入位置编码的目标就是希望额外保留原有文本的位置（索引）信息，

**什么样的位置编码是好的位置编码 –外推性好**

外推性好代表着如果你训练时最长文本是 1000，但真实推理时是 5000，也能很好适应.

**相对位置编码 *ROPE* 就有这样的特性**

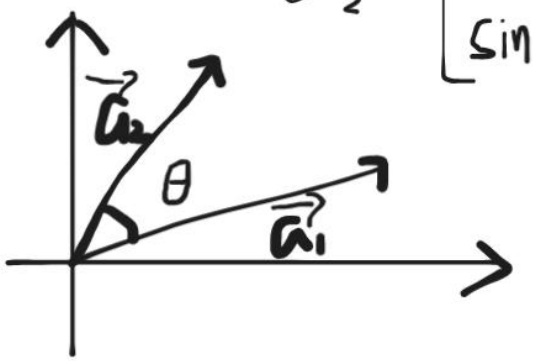
**什么是旋转矩阵？**

**旋转矩阵**是在乘以一个向量的时候改变了向量的方向但不改变大小的效果并保持了**手性**的矩阵。

**手性**指左手右手坐标系，类似物理上的左手定则，右手定则；

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} \qquad \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$

$$\vec{a}_2 = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \cdot \vec{a}_1$$




**性质**

设 M 是任何维的一般旋转矩阵:

- 两个向量的内积在它们都被一个旋转矩阵操作之后保持不变:
- 旋转矩阵的逆矩阵是它的转置矩阵:
- 若用 M(**θ**)表示逆时针旋转的角度为 **θ**，那么有 M(**α**+**β**) = M(**α**)M(**β**)
- 旋转矩阵的转置等于原矩阵角度取负，即 $\mathbf{M}(\boldsymbol{\theta}^T) = \mathrm{M}(-\boldsymbol{\theta})$

乘 M(α)代表先旋转α，再乘M(β) 代表再选转β，合计旋转了α+β，等效于乘 M(α+β)。

对于两个位置的 *token* 而言：

对于 $q\begin{pmatrix}q_1\\q_2\end{pmatrix}$，旋转 $m\theta$ 即为 $\begin{bmatrix}\cos m\theta & -\sin m\theta\\ \sin m\theta & \cos m\theta\end{bmatrix}\cdot\begin{bmatrix}q_1\\q_2\end{bmatrix}=\begin{bmatrix}q_1\cos m\theta-q_2\sin m\theta\\ q_1\sin m\theta+q_2\cos m\theta\end{bmatrix}=q_{-m}$

对于 $k\begin{pmatrix}k_1\\k_2\end{pmatrix}$ 同理旋转 $n\theta$ 得到 $k_{-n}\begin{bmatrix}k_1\cos n\theta-k_2\sin n\theta\\ k_1\sin n\theta+k_2\cos n\theta\end{bmatrix}$

计算 $q_{-m}$ 与 $k_{-n}$ 的点积，$(q_1\cos m\theta-q_2\sin m\theta)(k_1\cos n\theta-k_2\sin n\theta)$

$+$

$(q_1\sin m\theta+q_2\cos m\theta)(k_1\sin n\theta+k_2\cos n\theta)$

将其展开成 $q_i$ 与 $k_i$ 组合，

$$\Pi\ \sum\begin{cases}q_1\cdot k_1\cdot[\cos m\theta\cos n\theta+\sin m\theta\sin n\theta]\\ q_2\cdot k_2\cdot[\sin m\theta\sin n\theta+\cos m\theta\cos n\theta]\\ q_1\cdot k_2\cdot[\sin m\theta\cos n\theta-\cos m\theta\sin n\theta]\\ q_2\cdot k_1\cdot[\cos m\theta\sin n\theta-\sin m\theta\cos n\theta]\end{cases}$$

$\Downarrow$ 利用三角恒等式化简

$$(q_1\cdot k_1+q_2\cdot k_2)\cdot\cos(m-n)\theta+(q_1\cdot k_2-q_2\cdot k_1)\sin(m-n)\theta$$

其中 $q$、$k$ 皆可原始值，位置信息只由 $\begin{cases}\cos(m-n)\theta\\ \sin(m-n)\theta\end{cases}$ 决定

它们的位置信息只由相对的旋转角度决定，这就是旋转位置编码的思想-将每一个 token embedding 后的向量进行不同角度的旋转体现 m\n 上，在向量元素之间也进行角度旋转体现在$\theta_i$上。导致每一个位置的 token 都有自己专属的旋转角度，在注意力计算 Q 与$K^T$相乘时其对应的带有位置信息的 q、k 向量做点积实际上结果除 q、k 值只与相对旋转角度有关。通过相对旋转角度读取文本中 token 间的依赖关系而不受绝对的位置所限。

$$\begin{pmatrix}\cos m\theta_0 & -\sin m\theta_0 & 0 & 0 & \cdots & 0 & 0\\ \sin m\theta_0 & \cos m\theta_0 & 0 & 0 & \cdots & 0 & 0\\ 0 & 0 & \cos m\theta_1 & -\sin m\theta_1 & \cdots & 0 & 0\\ 0 & 0 & \sin m\theta_1 & \cos m\theta_1 & \cdots & 0 & 0\\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots\\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2-1} & -\sin m\theta_{d/2-1}\\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2-1} & \cos m\theta_{d/2-1}\end{pmatrix}\begin{pmatrix}q_0\\q_1\\q_2\\q_3\\\vdots\\q_{d-2}\\q_{d-1}\end{pmatrix}$$

对于词向量维度两两分组，看作 d/2 组二维向量做旋转矩阵乘法，其给**不同维度分配不同的旋转速度$\theta_i$**

$$\theta_i = \frac{1}{rope_base^{(\frac{2i}{dim})}}$$

i 越小，freqs 越大，即旋转角度沿位置方向变化的更快；i 越大，freqs 越小。所以低维度的快速旋转适合捕捉局部的位置关系，高维度的慢速旋转适合捕捉全局的位置关系。

原始 ROPE 在处理序列长度远超训练长度时的外推性不好：

高频维度 (High-frequency, 较小)： 频率很大，旋转很快。在训练窗口内，这些维度可能已经旋转了数十甚至上百圈。当长度超过训练长度时，其角度只是“再多转一点”（mod 后增多一点点），模型在训练中已经见过无数次相似的角度值，因此可以很好地处理外推。

低频维度 (Low-frequency, 较大)： 频率极小，旋转极慢。对于某些用于编码长距离关系的低频维度，其在接近训练长度时的总旋转角度可能仍小于 $2\pi$ 。当模型在推理时遇到超出训练长度的 token 时，这个维度的旋转角可能首次超过了 $2\pi$ 。

——OOD 灾难：模型在预训练期间从未见过该维度产生大于 $2\pi$的旋转角。它无法将这个 OOD 信号解释为有效的位置信息。这导致了不稳定的注意力分数，进而导致注意力机制的崩溃，特别是在评估长距离依赖关系时。

解决方案：

一：

位置插值：将需要处理的更长序列 （例如 32k）的 位置索引 （范围 ）线性缩放（或称“挤压”）到模型预训练的原始范围 （例如 4k）之内。但是他统一缩放了所有的维度，对于相邻很近的token其高频维度产生的旋转角度差异可能由于这个缩放因子过大变得微乎其微，导致模型难以分辨其顺序（高频信息损失）。

二：

Yarn：进行了分段处理，对于高频维度其实根本不应该进行插值，保留原频；对于低频部分完全应用缩放 1/s;过渡阶段进行线性插值。

$$i(b) = dim/2 \times \frac{ln(\frac{L}{2\pi b})}{ln(rope_base)}$$

$$low = max(i(\beta_{fast}),0), high = min(i(\beta_{slow}), dim/2 - 1)$$

先计算需要调整的维度的范围。定义函数 i(b),表示波长恰好等于 L/b 的维度索引，传入$\beta_{fast}$、$\beta_{slow}$调整插值区间的下界和上界，高频维度即 i<low 保留原始频率；低频维度即 i>high 完全应用缩放 1/s;low<i<high 的维度进行线性插值：

$$\gamma（\mathrm{i}） = \mathrm{clamp}(\frac{\mathrm{i-low}}{\mathrm{max(high-low,\epsilon)}}, 0,1)$$

再构建斜坡函数，对每一个维度索引定义插值权重

$$\theta_i' = \theta_i \cdot (1 - \gamma(i) + \frac{\gamma(i)}{s})$$

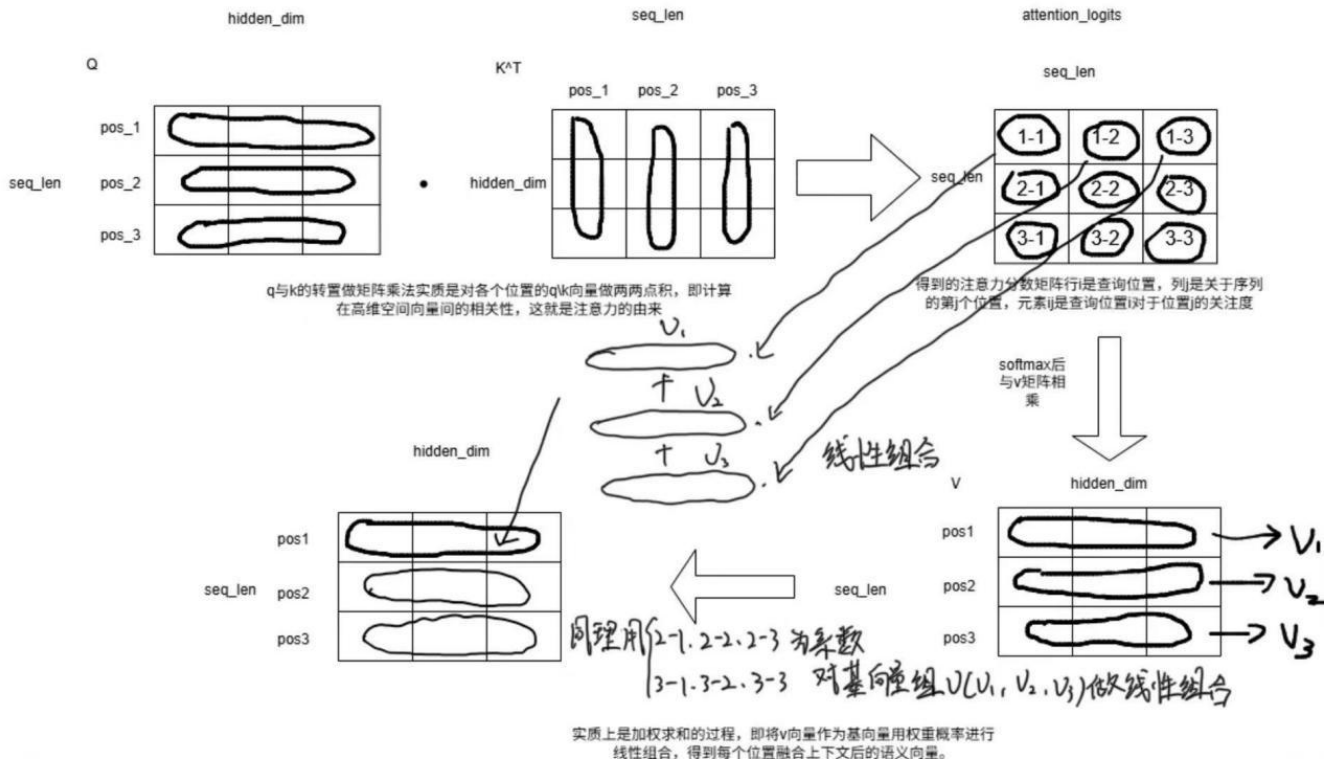
当γ（i）=0（高频部分），频率不变；

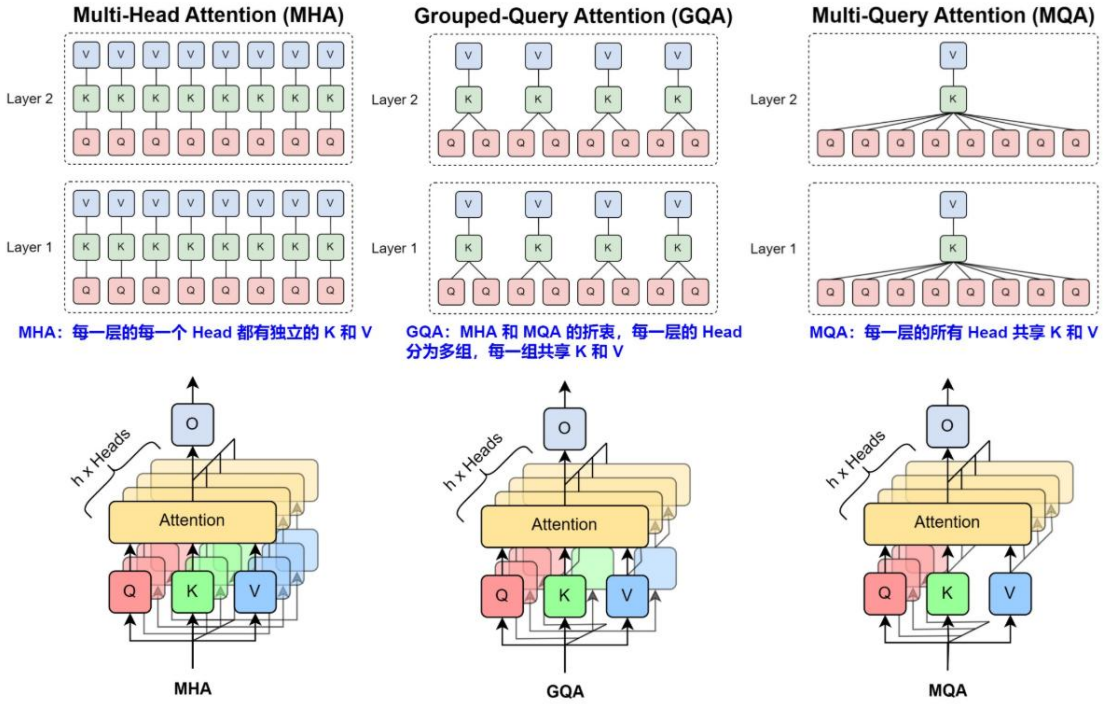
当γ（i）=1(低频部分)，$\theta_i' = \theta_i$/s，完全线性缩放

Attention:

对于一个序列的自回归生成，要理解其中的语义依赖和整体信息，需要一个 database 存放历史信息，并结合当下的输入生成对于下一个词的理解即预测。其中的 query 与 key 的倒置点乘实际上在计算向量的相似性（所谓的 atttention,更应该关注哪里），得到的向量各个维度的值实际上就是该输入对因果序列各个时刻的注意力分数，再进行维度缩放，softmax 归一化得到注意力权重向量，决定了在各个隐藏维度 dim 上对因果序列的 value 做加权求和即得到在各个维度上综合了历史信息，蕴含着对整个句子的理解的语义向量。（也可理解为各个时刻的 value 向量作为**基向量**进行**线性组合**，产生高维空间中一个特定方向的语义向量）

可以认为 q 是数据库中的查询，kv 构建了历史状态的语义库；






在 MHA(muti-head attention)计算中往往划分不同查询头代表对不同维度的语义信息的理解，即把 Q、K、V 分别拆分成多份，每个注意力头使用独立的 Q、K、V 进行计算。而多个头可以并行计算，分别得出结果，最后再合回原来的维度。

$$\mathbf{h}_t \in \mathbb{R}^d \xrightarrow[\text{生成Q、K、V}]{1} \begin{aligned} \mathbf{q}_t &= W^Q \mathbf{h}_t, \\ \mathbf{k}_t &= W^K \mathbf{h}_t, \\ \mathbf{v}_t &= W^V \mathbf{h}_t, \end{aligned} \xrightarrow[\text{拆分}]{2} \begin{aligned} [\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; \ldots; \mathbf{q}_{t,n_h}] &= \mathbf{q}_t, \\ [\mathbf{k}_{t,1}; \mathbf{k}_{t,2}; \ldots; \mathbf{k}_{t,n_h}] &= \mathbf{k}_t, \\ [\mathbf{v}_{t,1}; \mathbf{v}_{t,2}; \ldots; \mathbf{v}_{t,n_h}] &= \mathbf{v}_t, \end{aligned}$$

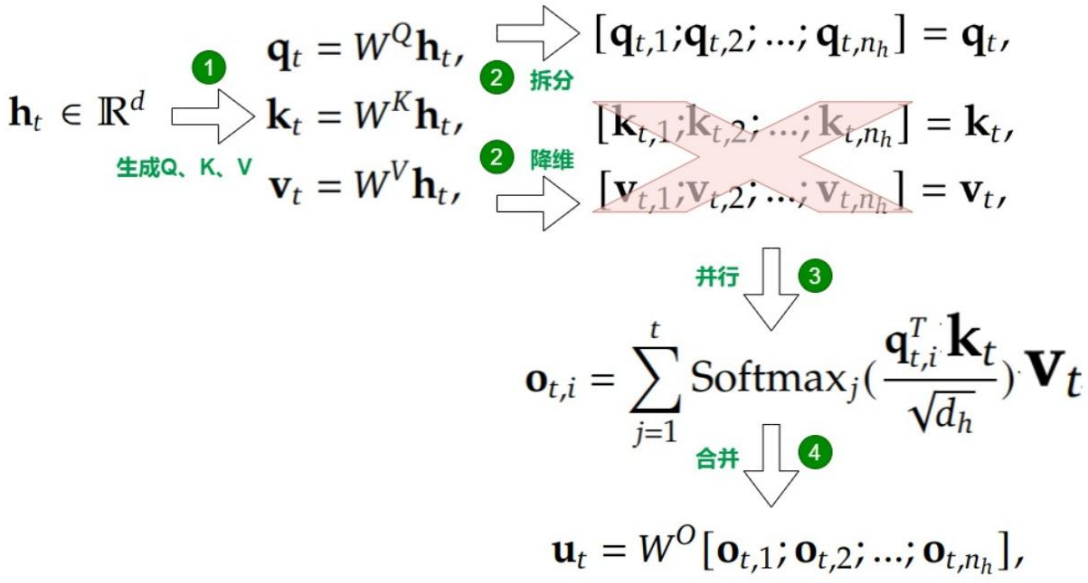
$$\xrightarrow[\text{并行}]{3}$$

$$\mathbf{o}_{t,i} = \sum_{j=1}^{t} \mathrm{Softmax}_j\left(\frac{\mathbf{q}_{t,i}^T \mathbf{k}_{j,i}}{\sqrt{d_h}}\right) \mathbf{v}_{j,i},$$

$$\xrightarrow[\text{合并}]{4}$$

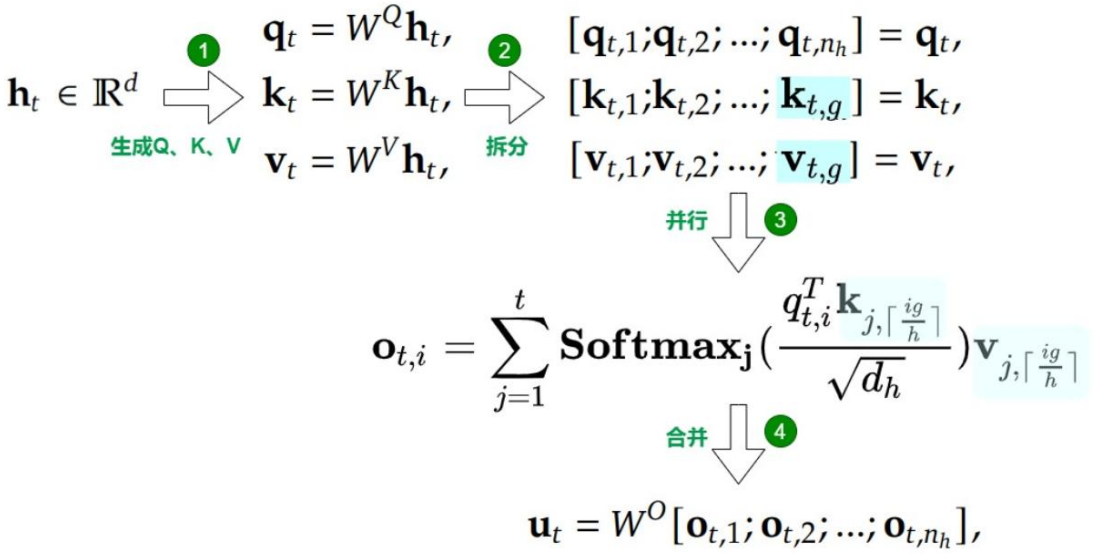
$$\mathbf{u}_t = W^O [\mathbf{o}_{t,1}; \mathbf{o}_{t,2}; \ldots; \mathbf{o}_{t,n_h}],$$

对于 MQA：只对 Q 进行切分,所有的 Query 头共用同一组 K 和 V。从而减少了 kv cache 中的显存开销，开销为原先的 1/n,n 是查询头的个数。




在输入向量进行线性投影的时候就已经将输出的k\v维度限制为划分的一个查询头的维度，但这样显然损耗了一些信息，对于文本理解能力有一定损害。

对于 GQA，是 MHA 和 MQA 的 trade off,取了一个折中方案令每 g 个查询头共享一个 k\v 矩阵，显存开销为原始 MHA 的 g/n,n 为查询头总数，同时存储的信息量、文本的理解能力都比 MQA 好。




## ResNet:

当某一层的输出为 H(X),其需要拟合的数据分布为 F（x）,则需要令 H（x）=F(X),较为吃力。当引入残差时：就需要令 H(x)+x=F(x),此时 H(X)只需拟合成 F(x)-x,往往当 x 中即过往的特征中已经与待拟合的 F（x）相近时，F(X)-X 的拟合难度将会很小，引入的残差相当于“站在了输入的肩膀上”而不用“从头重新拟合”一个复杂的映射。尤其是在深层神经网络，在较深的 layer 往往相邻两层对数据分布的改变只是在微调，加入残差能令训练的效果稳定，

且能有效缓解梯度消失问题：

设引入残差后 layer 的输出为 Y,该层的输出为 F(x),则有 Y=F(x)+x:

$\frac{dL}{dx}=\frac{dL}{dY}\times\frac{dY}{dx}=\frac{dL}{dY}\times(\frac{dF}{dx}+1)$,防止由于$\frac{dF}{dx}$ *接近于* 0 *造成的梯度消失问题*

## 激活函数：

如果没有激活函数，多层神经网络在数学上等同于单层线性模型。线性函数的复合依然是线性的：

假设 f(x) = ax + b。如果堆叠 100 层这样的线性层，最终的结果依然可以简化为 F(x) = Ax + B。

引入非线性，相当于弯曲曲线使其拟合能力增强，神经网络中的一层层的 layer 各自拟合最终想得到的数据分布的部分曲线( （Universal Approximation Theorem）： 只要有足够多的神经元和至少一个非线性激活层，神经网络理论上可以拟合任何复杂的连续函数)，通过逐层累积最终得到近似地所需的复杂映射关系

万能近似定理：

对于任意给定的一个函数 F(x)，存在一个整数 $M$,和一组实数$v_m$、$b_m$,以及实数向量 $W_m$,$m$ = 1, ⋯ , $M$，连续有界的单调递增函数$\phi(x)$,以至于我们可以定义函数:

$$F(\boldsymbol{x})=\sum_{m=1}^{M}v_m\phi(\boldsymbol{w}_m^{\mathsf{T}}\boldsymbol{x}+b_m),$$

对于只有一层隐藏层的神经网络而言，只要给定足够多的满足要求的神经元和恰当的权重配比，就能拟合出任意想要的映射关系。

$$\boldsymbol{z}^{(l)}=\boldsymbol{W}^{(l)}\boldsymbol{a}^{(l-1)}+\boldsymbol{b}^{(l)},$$

$$\boldsymbol{a}^{(l)}=f_l(\boldsymbol{z}^{(l)}).$$

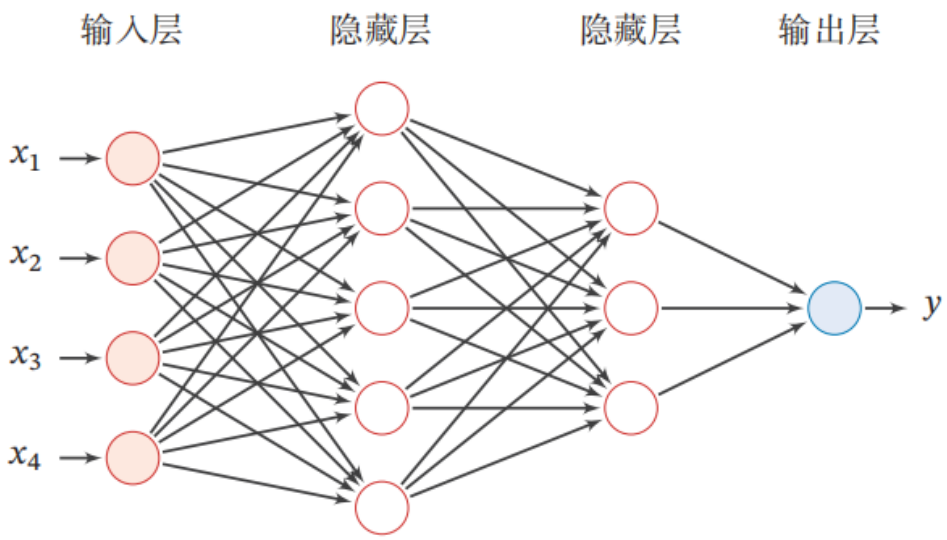
$$\boldsymbol{z}^{(l)} = \boldsymbol{W}^{(l)} f_{l-1}(\boldsymbol{z}^{(l-1)}) + \boldsymbol{b}^{(l)},$$

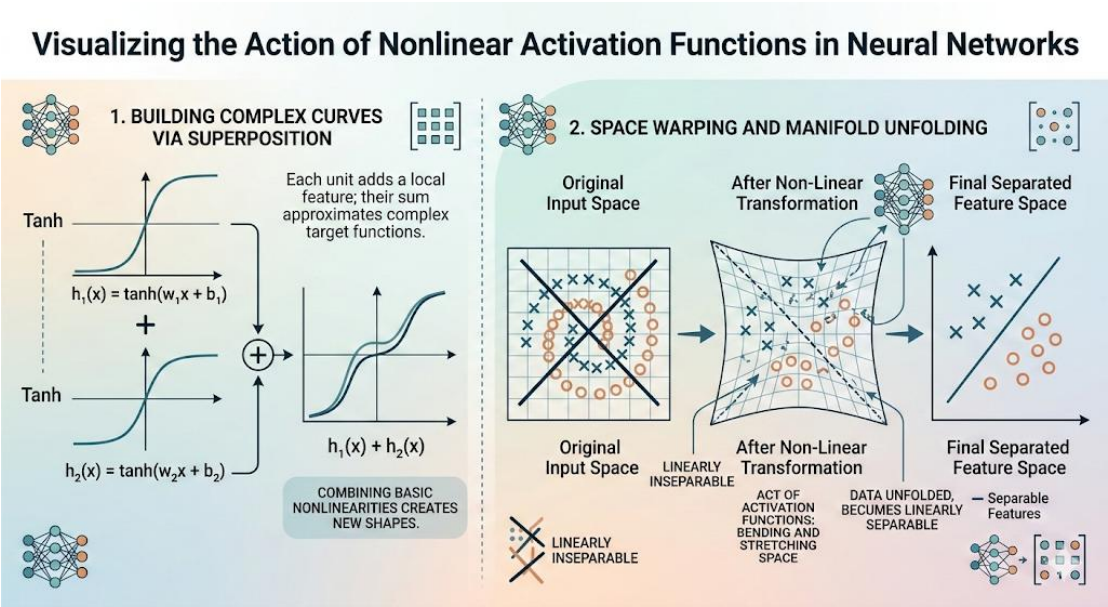
$$\boldsymbol{a}^{(l)} = f_l(\boldsymbol{W}^{(l)} \boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)}).$$

可见隐藏层先对输入向量进行仿射变化再输入到激活函数中，最后按照权重配比输出到输出层，每一个 layer 即可当作一个万能的函数近似。在深层神经网络中，不同层间拟合出不同的映射，逐层提取抽象化特征，使得模型拥有强大的特征表达能力，能得到深刻的理解能力和拟合数据分布能力。

$$h^{(l)} = f^{(l)}(h^{(l-1)}) = \sigma(W^{(l)} h^{(l-1)} + b^{(l)})$$

$$\hat{y} = f^{(L)}(f^{(L-1)}(\ldots(f^{(1)}(x))\ldots))$$

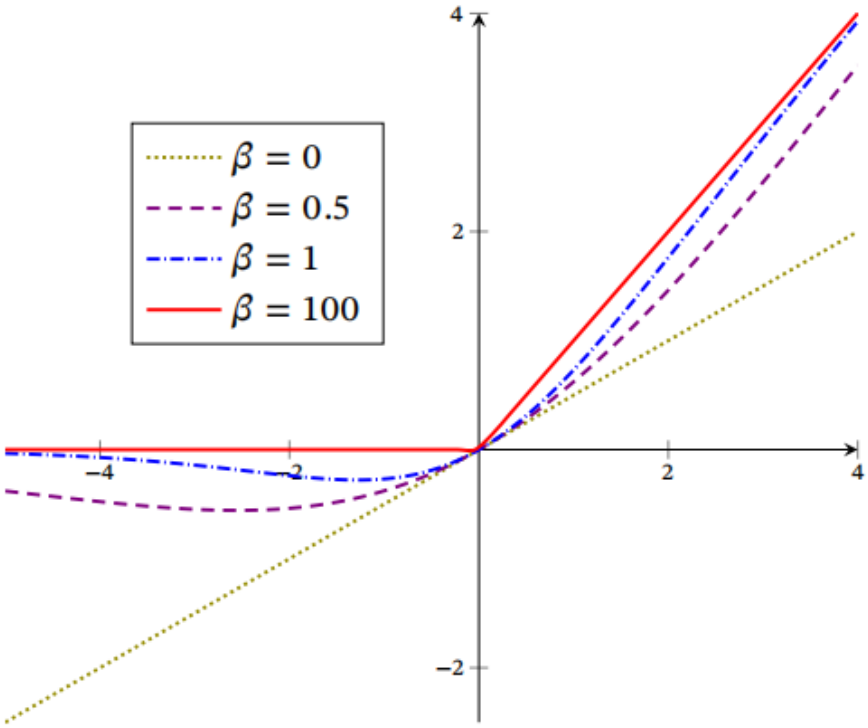





SILU 激活函数：

f(x)=$x \cdot \sigma(x) = \frac{x}{1+e^{-x}}$

其为 swish：$x \cdot \sigma(\beta x)$当 β 取值为 1 时的情况:




在 0 附近，它允许一部分负值信息流过，这增强了网络对微小信号的捕捉能力,能够保留一些微弱的负向特征，有助于提高模型的泛化能力.

当 x 很大时几乎是 y=x 直线，x 是很小的负值的时候几乎为 0

Sigmoid 型函数：

1.logistic 函数：

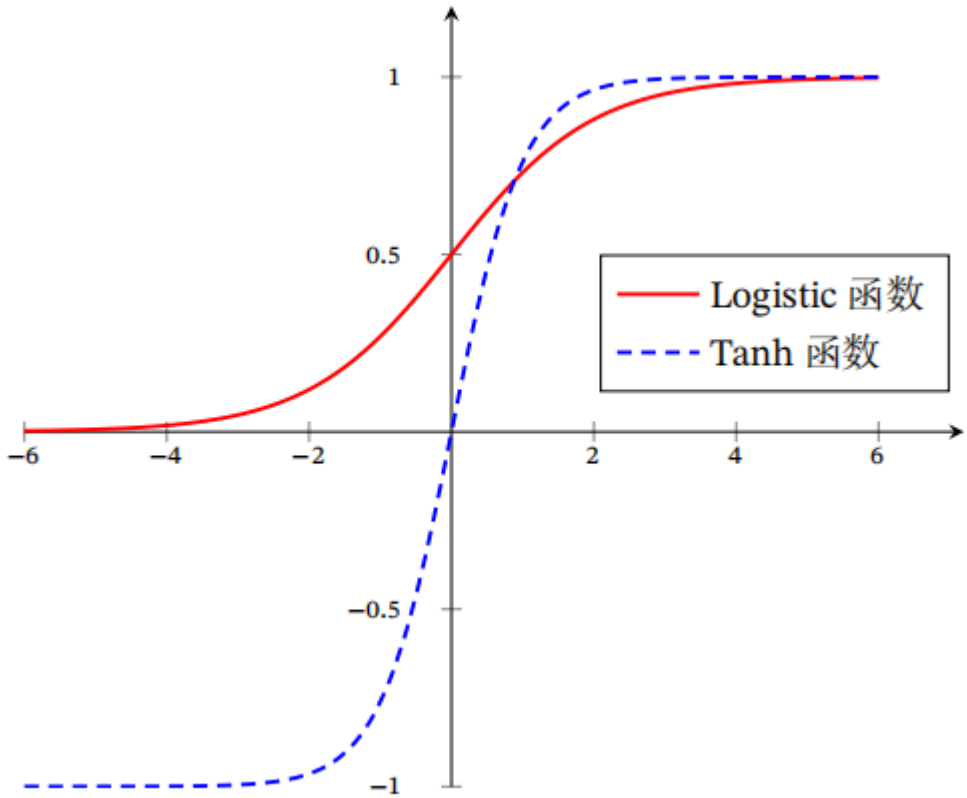
$$\sigma(x) = \frac{1}{1+e^{-x}}$$

可以看成是一个“挤压”函数，把一个实数域的输入“挤压”到 (0, 1)．当输入值在 0 附近时，Sigmoid 型函数近似为线性函数；当输入值靠近两端时，对输入进行抑制．输入越小，越接近于 0；输入越大，越接近于 1．

2. tanh 函数：

$$\text{Tanh}(x)=\frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$$

Tanh 函数的输出是零中心化的（Zero-Centered），而 Logistic 函数的输出恒大于 0




RELU 函数：

ReLU（修正线性单元），实际上是一个斜坡函数，定义为 $\text{relu}(x) = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}$

优点:

采用 ReLU 的神经元只需要进行加、乘和比较的操作，计算上更加高效．
在优化方面，相比于 Sigmoid 型函数的两端饱和，ReLU 函数为左饱和函数，

且在 $x>0$ 时导数为 1，在一定程度上缓解了神经网络的梯度消失问题，加速梯度下降的收敛速度.

缺点 ：

ReLU 函数的输出是非零中心化的，给后一层的神经网络引入偏置偏移,会影响梯度下降的效率.

ReLU 神经元指采用 ReLU 作为激活函数的神经元. ReLU 神经元在训练时比较容易“死亡”.在训练时，如果参数在一次不恰当的更新后，第一个隐藏层中的某个 ReLU 神经元在所有的训练数据上都不能被激活，那么这个神经元自身参数的梯度永远都会是 0，在以后的训练过程中永远不能被激活. 这种现象称为死亡 ReLU 问题,并且也有可能会发生在其他隐藏层.

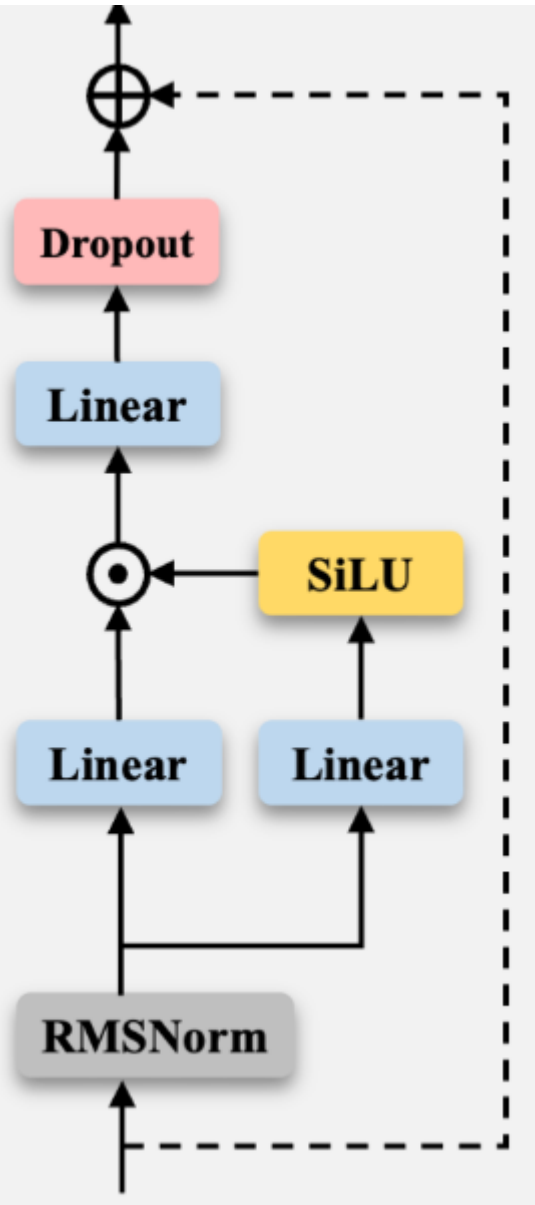
## FFN：

对于 attention 层读取到的语义向量进行更高纬度的解读、更细节化的扩展、更深层的理解。

首先对张量最后一维升维使其投影到更高维度的语义空间中，相当于对原先浓缩的信息进行舒展剖析，将原本一个维度运用多个子维度进行描述，挖掘出更细节的信息。

门控单元：两条并行计算路径，一条用 SiLU 激活函数做门控信号，另一条保持线性变换进行升维，最后把两条路径的结果相乘。模型可以自主调节每个神经元的通行权重。其作用是完成对输入文本的表征，通过门机制控制信息通过的比例，来让模型自适应地选择哪些单词和特征对预测下一个词有帮助。

接着将门控单元输出进行降维，投影回隐藏层的维度空间，相当于进行知识的提炼浓缩，最后通过 dropout 提升泛化能力




## KV cache：

由于自回归生成模型的因果性（过去的 token 信息不变、过去的 token 用不上后面的信息），在增量推理时，把每一层 Attention 中、每个已生成 token 对应的 K 向量和 V 向量缓存下来。后续生成新 token 时，只需要计算新 token 自己的 Q、K、V，然后将新的 K、V 追加到缓存中，用缓存里的完整 K、V 序列做 Attention。从而使得不必重复计算之前 token 不变的 k\v。减少的计算时间实际上牺牲了存储空间，当序列过长时会带来巨大的显存压力。

伪代码：

```
kv_cache = {}   # 每一层缓存 K, V
for step in range(max_new_tokens):
```

```
if step == 0:
    # 第一步：处理所有 input tokens，填充 cache
    logits, kv_cache = model(input_tokens, kv_cache=None)
else:
    # 后续步：只送入上一步生成的 1 个 token
    logits, kv_cache = model([last_token], kv_cache=kv_cache)
next_token = sample(logits[-1])
last_token = next_token
```

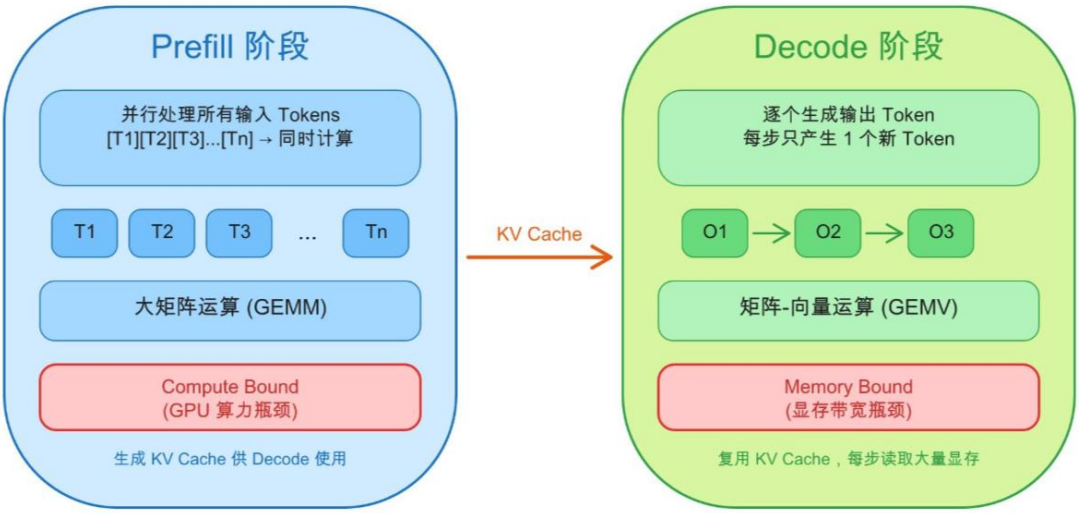
伪代码中 step==0 的 case 实际上对应着 Prefill 阶段：

模型一次性处理所有输入 token（system prompt + user message），为每一层、每个 token 计算出 K 和 V 并存入 cache。这一阶段处理 token 是并行的（大矩阵计算，运用因果掩码），由于一次性处理大矩阵计算，性能瓶颈是 GPU 算力。

【prefill 阶段既然是推理，看上去只需要计算最后一个 token 位置的预测？】
——【transformer 有许多层（block），需要逐层抽象，上一层的输出当作下一层的输入，所以需要计算每个位置一直到最后一层】

else 即后续的 step 对应着 Decode 阶段：

每一步只输入 1 个 token，利用 KV Cache 做 Attention，生成下一个 token。增量推理阶段由于 token 之间存在依赖关系，无法并行只能每步处理 1 个 token，每步计算量小但都需要从显存中读取 kv cache ，IO 开销大。




## DataSet:

数据集中数据的格式为 json-l,即一行就是一个完整的 json 文本，不仅对内存友好且方便分割。

Dataset 是 PyTorch 中用于数据集抽象的类。自定义数据集需要继承 torch.utils.data.Dataset 并重写以下两个方法：
__len__：返回数据集的大小。
__getitem__：按索引获取一个数据样本及其标签。

## 动态学习率：

可以帮助模型在不同阶段使用不同的学习速度，以避免在训练早期学习率过小收敛速度过慢或在训练后期学习率过大在最优解两侧来回震荡的问题。

余弦退火学习率：

$$\eta_t = \eta_{min} + \frac{(\eta_{max} - \eta_{min}) \cdot (1 + cos(\frac{\pi \cdot T_{cur}}{T_{max}}))}{2}$$

| 符号 | 名称 | 解释 |
|---|---|---|
| $\eta_t$ | 当前学习率 | 在当前迭代时刻（步数或轮数）计算出的实际学习率。 |
| $\eta_{max}$ | 最大学习率 | 学习率下降的起点，通常也是设定的初始学习率。 |
| $\eta_{min}$ | 最小学习率 | 学习率下降的终点，通常设为一个非常小的值（如 0）。 |
| $T_{cur}$ | 当前步数 | 当前训练到了第几个 Epoch 或第几个 Batch。 |
| $T_{max}$ | 最大步数 | 预设的一个周期（Cycle）的总步数。 |

该公式核心在于非线性部分$cos(\frac{\pi \cdot T_{cur}}{T_{max}})$,当训练开始时$T_{cur} = 0$,余弦值为 1，此时公式简化为$\eta_{max}$ ；当训练到达预设终点时，$T_{cur} = T_{max}$ ,余弦值为-1，此时公式简化为$\eta_{min}$

学习率并不是线性下降，而是遵循余弦曲线：
前期：下降速度较慢，保持较大波动以跳出局部最优。

中期：下降速度加快。

后期：下降速度再次放缓，在极小值附近进行微调，使模型收敛更稳定。

通过模拟余弦函数的曲线，让学习率在训练过程中平滑地下降，从而帮助模型在初期快速收敛，并在后期更精细地寻找全局最优解

## 梯度累积：

GPU 显存是有限的，当其不能满足大 batch 的训练时，可以通过小批次训练梯度累积来模拟大批此训练

当显存 batch 为 n 时，要模拟 batch 为 N 时:先连续进行（N/n）次前向和反向传播，将计算出的梯度相加起来，用这个梯度进行参数更新再清空梯度.对于每一个 batch 前向传播算的损失都需要除以（N/n）来平均化损失。

## 混合精度：

在计算过程中，同时使用 16 位浮点数 (FP16) 和 32 位浮点数 (FP32)。

- FP32 (32-bit)：精度高，范围广，但占用显存多，计算慢。
- FP16 (16-bit)：占用显存减半，计算速度极快，但精度低，容易溢出（数值太大变 NaN）或舍入误差（数值太小变 0）。

混合精度取长补短：用 FP16 跑耗时的矩阵运算，用 FP32 保存关键权重：

① 权重备份 (Master Weights)

模型在更新时，会维护一份 FP32 的权重副本。计算时：将权重转为 FP16 进行前向和反向传播；更新时：将计算出的 FP16 梯度加回到 FP32 的权重副本上。

② 损失缩放 (Loss Scaling)

这是解决 “梯度消失” 的关键。FP16 能表示的最小正数有限，很多梯度往往比这个还小，直接变 0。

做法：在反向传播前，将 Loss 乘以一个很大的系数（比如 1024），把微小的梯度“放大”到 FP16 的有效范围内。结尾：在更新权重前，再把梯度除以同样的系数还原回去。

## Flash attention:

### 为什么需要 flash attention?

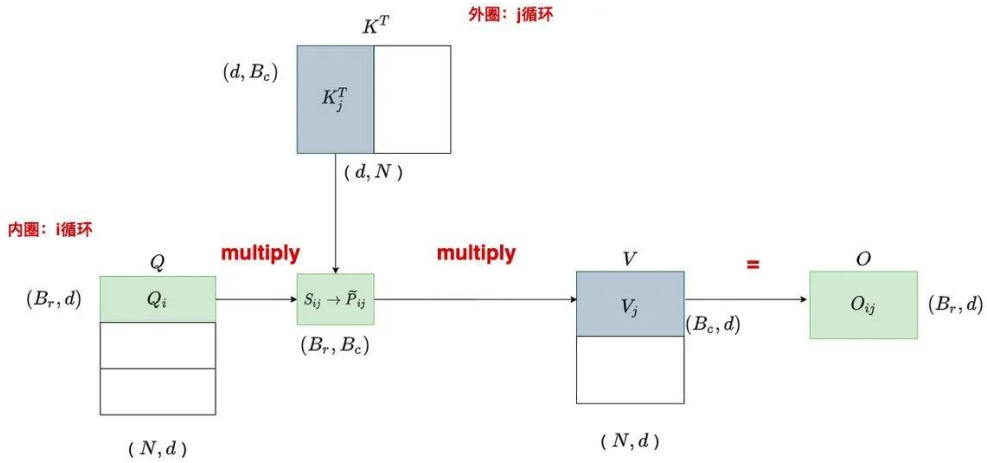
在 attention 计算中，由于实质上是进行庞大的矩阵运算，会产生频繁、巨大的对 gpu 显存的读写开销（由 attention 计算过程中产生的中间矩阵产生，如未 softmax 的注意力分数矩阵、softmax 后的概率矩阵等等，这些中间矩阵被反复从显存中读入读出）。

Flash attention:

为了减少大量的 IO 开销，减少中间结果的读入读出操作，并利用片上缓存的计算高速性。采取了先对 q\k\v 矩阵进行分块，将小块读入片上高速缓存，在其中计算部分的 attention 最终结果，并通过**保存状态变量**进行**在线更新**（softmax 需要“看到”整体），最终将整体的 attention 最终结果写入显存，这样就避免读到中间的 N·N 的大矩阵（序

列通常非常长，维度 d 相比之下就显得只是常数项的量级，即优化成只读取原始 q\k\v，即 N· $d$的复杂度，为线性复杂度）

利用高速缓存分块运算




- 它将输入的查询（Q）、键（K）、值（V）矩阵从 HBM 中切分成块

- 然后，它逐块将这些数据加载到高速的 SRAM 中 。所有的中间计算步骤，包括矩阵乘法和 Softmax，都在 SRAM 内部完成，完全不产生将巨大的 ‘N×N‘中间矩阵写回 HBM 的操作。

- 最后，只将计算完成的最终输出结果从 SRAM 一次性写回 HBM 。

关键的中间的在新更新算法：

FlashAttention 的核心思想是：我们可以迭代地更新 Softmax 的计算结果。每当一个新的数据块到来时，我们用它来修正之前基于旧数据块计算出的（不完整的）结果。

$softmax(x_i) = \frac{e^{x_i - m}}{\sum_{j=1}^{n} e^{x_j - m}}$ ,其中 m 是某行\列的最大值（为了数值稳定性）

让我们把注意力计算中的一行（例如，$Q_i$ 和所有 $K_j$ 的点积）看作是我们要计算 Softmax 的输入向量 $x$。这个向量 $x$ 被分成了多个块 $x1, x2, ..., xc$

算法为这一行维护三个核心变量：

$O$: 当前的输出（对 V 的加权和）。

$m$: 到目前为止见过的所有 $x$ 元素中的最大值（running maximum）。

$l$: 到目前为止 Softmax 分母的累加和（running denominator）。

模拟算法的迭代过程：

假设我们已经处理了第 1 到 j-1 个块，得到了当前的统计量 $m_{old}$ 和 $l_{old}$，以及当前的输出 $O_{old}$。

现在，我们处理第 j 个块，$x_j$:

**1. 计算当前块的局部统计量**

我们只看当前块$x_j$ ，可以计算出它的：

局部最大值: $m_j = max(x_j)$

局部 Softmax 分子: $P_j = e^{x_j - m_j}$

局部 Softmax 分母: $l_j = sum(P_j)$

**2. 合并统计量，更新全局最大 ：**

新的全局最大值，一定是旧的全局最大值和当前块最大值中的较大者：$M_{new} = max(m_{old}, m_j)$

**3 .重新缩放（Re-scaling）并更新分母 $l$：**

这是最关键的数学技巧。我们之前的 $l_{old}$ 是基于 $M_{old}$ 计算的，而 $l_j$ 是基于 $M_j$ 计算的。现在我们有了新的全局最大值 $M_{new}$，我们需要把这两部分都统一到新的基准上再相加。

$$l_{new} = l_{old} * e^{m_{old} - m_{new}} + l_j * e^{m_j - m_{new}}$$

这是显然的

**4. 重新缩放并更新输出$O$**

输出 $O$ 是对值矩阵 $V$ 的加权和。它的更新逻辑和分母 $l$ 完全一样，也需要重新缩放。

$$O_{new} = \frac{O_{old} * l_{old} * e^{m_{old} - m_{new}} + p_j * v_j * e^{m_j - m_{new}}}{l_{new}}$$

显然前半段是对旧值的修正，后半段是新值加入

## Pre-train:

预训练步骤：

1. 从 dataset 获取 input_ids 和 labels，attention mask
2. 前向传播，计算 loss，反向传播计算梯度，梯度下降更新参数。

## SFT（有监督微调）：

将做无监督的文本接龙，学会语言的统计规律和世界知识的 base 模型进阶为能遵从指令回答和会话范式的助手模型。

与 pre-train 不同 SFT 的训练并非直接将整个文本当作上下文输入关心所有 token 的预测，而是会区分指令和回答将其嵌入到对话模板中，即输入指令部分让模型预测回答，将标准答案作为 label 计算 loss,这里会对指令部分进行掩码即不计算指令部分的 loss（指令部分是人类提供的已知信息，如果计算这部分的 Loss，模型会尝试去“背诵”这些问题，这不仅浪费计算资源，还会导致模型过拟合于特定的提问方式）。模型还必须学会何时停止（预测出 Stop Token）。

典型的 Prompt 结构： [Round 1]

User: {Instruction}

Assistant: {Response}<|endoftext|>

为了避免较大的破坏模型预训练学习到的参数，SFT 阶段的学习率会较小，为了只在原来基础上进行微调参数。